

CHAPTER 22

Sun-nadir pointing control

22.1. Space story

When we worked on GPS IIR at GE Astro Space, I do not think anyone had an idea of the impact it would have on society. As it was a US Air Force program, we thought mostly of military applications. The idea that millions, if not billions of people would someday be traveling with hand-held phones using GPS to find their way was not anything we imagined.

22.2. Introduction

This chapter provides the theory behind Sun-nadir pointing-control systems. A Sun-nadir pointing control system is designed to point one axis of the spacecraft at the Earth and keep the solar-array cell face normal aligned with the Sun vector. This maximizes power input while keeping the Earth-pointing payload on target. For GPS, this would be the GPS antennas. Sun-nadir pointing-control systems are generally used on spacecraft in highly inclined orbits where the changes in the Sun angle with respect to the orbit plane may be large.

A characteristic of Sun-nadir pointing spacecraft is that its yaw angle changes over the orbit. When the Sun angle is near the orbit plane the spacecraft flips nearly 180° at spacecraft noon and midnight. Since the spacecraft must rotate in pitch and yaw, this precludes the use of a momentum bias or spinning spacecraft.

This chapter deals only with Sun-nadir pointing control. Station-keeping and acquisition modes are not covered. In addition, the accompanying scripts do not include logic for reinitializing of the control loops, error handling, and other necessary functions that must be included in flight-control software.

22.3. Coordinate frames

The orbit geometry is illustrated in Fig. 22.1. The Sun angle is defined as positive above the orbit plane and its projection in the orbit plane is along the $+X$ -axis. The spacecraft vector in the orbit plane is an angle α away from the Sun projection. The local vertical/local horizontal (LVLH) frame is defined with $+z$ along the nadir vector, $+y$ along the orbit antinormal, and $+x$ completing the right-hand set. The spacecraft is yawed an angle θ with respect to the LVLH frame. Finally, the solar arrays are oriented at an angle γ with respect to the body frames. When $\gamma = 0$ the $+z$ array axis (the cell face)

is aligned with the $+z$ body axis. In this chapter, the inertial frame will be defined with the XY -axes in the orbit plane, the $+Z$ -axis along the orbit normal and the $+X$ -axis aligned with the Sun projection. This will not normally coincide with standard inertial frames such as the ECI or J2000.0 frames. The solar array frame is illustrated in Fig. 22.2.

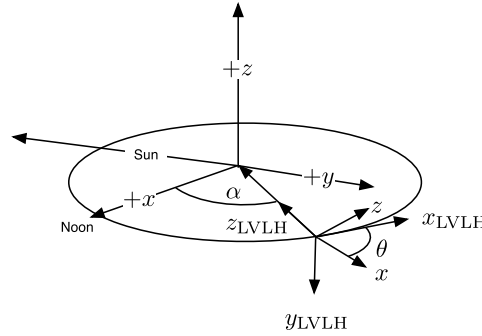


Figure 22.1 Orbit-geometry diagram.

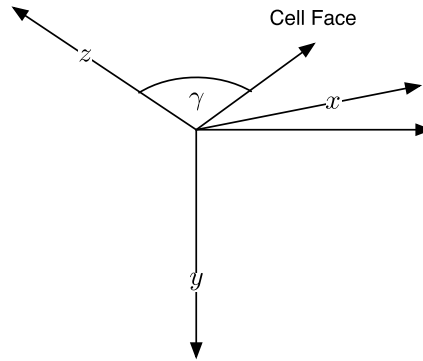


Figure 22.2 Solar-array frame.

22.4. Sun-nadir pointing

The Sun vector is represented by

$$u_s = \begin{bmatrix} -\sin \alpha \cos \beta \\ -\sin \beta \\ -\cos \alpha \cos \beta \end{bmatrix} \quad (22.1)$$

where β is the elevation angle of the Sun and α is the azimuth. The transformation matrix from the array to the core frame is

$$\begin{bmatrix} \cos \gamma & 0 & \sin \gamma \\ 0 & 1 & 0 \\ -\sin \gamma & 0 & \cos \gamma \end{bmatrix} \quad (22.2)$$

where γ is the rotation angle of the solar array. The $+z$ unit vector in the array frame is, in the core frame,

$$u = \begin{bmatrix} \sin \gamma \\ 0 \\ \cos \gamma \end{bmatrix} \quad (22.3)$$

The transformation matrix from the core frame to the LVLH frame is

$$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (22.4)$$

Transforming into the LVLH frame, assuming a yaw rotation of θ , and equating to the Sun vector gives

$$\begin{bmatrix} -\sin \alpha \cos \beta \\ -\sin \beta \\ -\cos \alpha \cos \beta \end{bmatrix} = \begin{bmatrix} \cos \theta \sin \gamma \\ \sin \theta \sin \gamma \\ \cos \gamma \end{bmatrix} \quad (22.5)$$

Therefore, the yaw and solar array angles are

$$\theta = \tan^{-1}(\sin \beta, \sin \alpha \cos \beta) \quad (22.6)$$

$$\gamma = \tan^{-1}(-\sqrt{(\sin \alpha \cos \beta)^2 + \sin^2 \beta}, -\cos \alpha \cos \beta) \quad (22.7)$$

The two argument $\tan^{-1}$ is usually called atan2 and puts the vector in the correct quadrant in all cases. Fig. 22.3 shows the Sun-nadir trajectories.

The plots show trajectories ranging from $\beta = 0^\circ$ to $\beta = 70^\circ$. When $\beta = 0$, the yaw angle is always zero and the array rotates 360° every orbit. For $\beta = \pm\epsilon$, the yaw angle changes 180° at spacecraft noon and midnight. If reaction wheels are used to perform the noon-midnight turns, the peak yaw rate and yaw inertia will largely determine the momentum requirements of the system. Because of limitations in the momentum storage (or control authority if using thrusters), it will not be possible to track the ideal yaw trajectory exactly.

Assuming that the solar arrays can always be pointed so that the only error is in the γz -plane of the solar arrays, the dot product of the Sun vector in the solar-array plane and the solar-array normal will be

$$\sqrt{1 - u_y^2} = \cos \epsilon \quad (22.8)$$

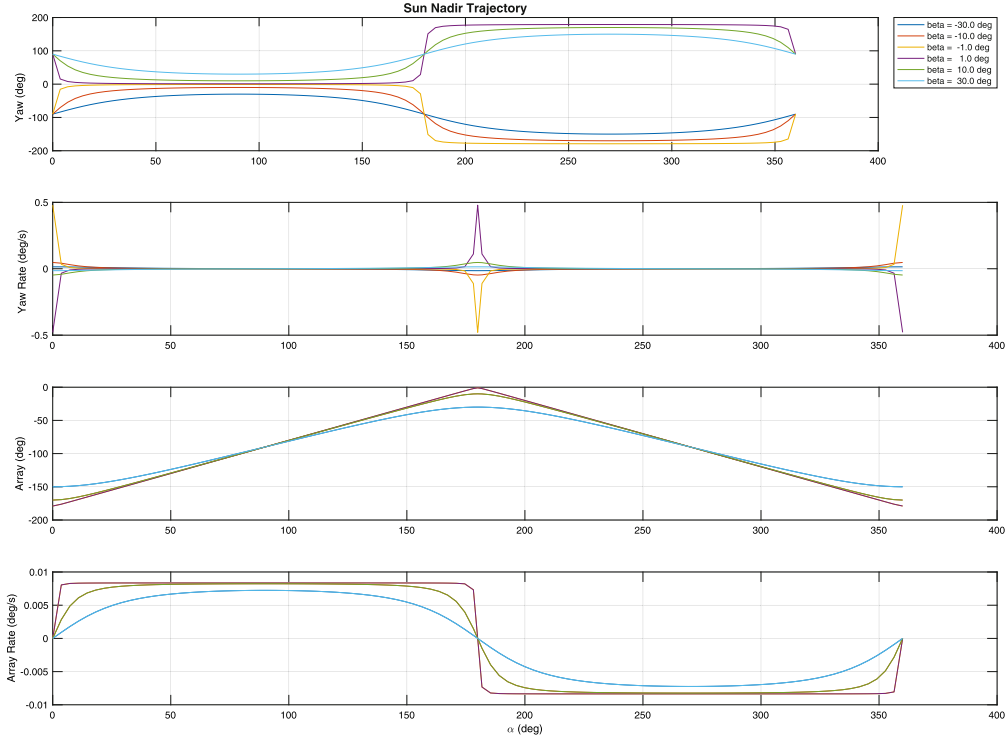


Figure 22.3 Sun-nadir yaw trajectory for a GPS orbit. The noon–midnight flips are evident.

$$1 - u_y^2 = \cos^2 \epsilon \quad (22.9)$$

$$u_y = \sin \epsilon \quad (22.10)$$

where u_y is found by transforming the Sun vector in the LVLH frame to the core frame; therefore

$$\sin \theta \sin \alpha \cos \beta - \cos \theta \sin \beta = \sin \epsilon \quad (22.11)$$

$$\theta - \theta_{ideal} = \sin^{-1} \left(\frac{\epsilon}{\sin \alpha} \right) \quad (22.12)$$

The second equation is found using the relationship

$$A \sin(a + b) = a \sin a \cos b + A \cos a \sin b \quad (22.13)$$

and gives us the relationship between angular error and deviation from the ideal trajectory. This can be implemented by feeding an offset as a function of α and β into the

yaw-error measurement. When β is small this becomes

$$\theta - \theta_{ideal} = \sin^{-1} \left(\frac{\epsilon}{\sqrt{(\sin \alpha \cos \beta)^2 + \sin^2 \beta}} \right) \quad (22.14)$$

or

$$\epsilon = \sin \alpha \sin(\theta - \theta_{ideal}) \quad (22.15)$$

which implies that even large errors at noon and midnight cause only small errors in the solar-array pointing.

22.5. Components

22.5.1 Sensors

Three-axis sensing is required for a Sun-nadir pointing spacecraft. One configuration is to measure roll and pitch with an Earth sensor and yaw with a Sun sensor. If the Sun sensor is array mounted, three-axis information will be available except in a narrow region about spacecraft noon and midnight. The sensing gaps can be bridged with gyros or (since as discussed above, the solar-array pointing error becomes less sensitive to yaw error as the spacecraft approaches the noon/midnight regions) we can rely on ephemeris information to estimate yaw.

Another possibility is to use a star sensor for all three axes and rely on ephemeris information to determine the orientation of the nadir vector. For very precise applications, the star sensor could be used in conjunction with gyros. For less-precise pointing, the star sensor could be used by itself. If gyros are used, a bright star sensor may be appropriate.

In this chapter, the combination of a conical scanning Earth sensor and array-mounted Sun sensors will be used. This is a relatively low-cost option and should provide adequate pointing for a GPS satellite-type application (0.1° in roll and pitch and 3° solar-array pointing error).

Four Sun-sensor heads are mounted on the arrays, two on each array. The sensor geometry is illustrated in Fig. 22.4. One sensor's boresight is in the γz -array plane the other is in the xz -array plane. The first is canted 45° from γ and the other 45° from $+z$. The elements on the other array are canted in the opposite direction so that the γz sensor points down (in the picture) and the xz sensor points to the left. The Earth sensor has a motor that rotates a mirror or prism that sweeps the field of view of the sensing element across the Earth, see Fig. 22.5. Pitch is found by measuring the center of the chord with respect to an internal reference and roll is found from the length of the chord. If the scan angle is set properly the scan will traverse the northern or southern hemisphere of the Earth resulting in a one-to-one relationship between chordwidth and attitude. This will not be the case for large attitude errors.

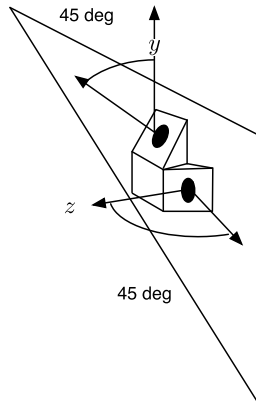


Figure 22.4 Sun-sensor geometry. These sensors are used to determine yaw. y is along the solar-array rotation axis. $+z$ is normal to the cell face.

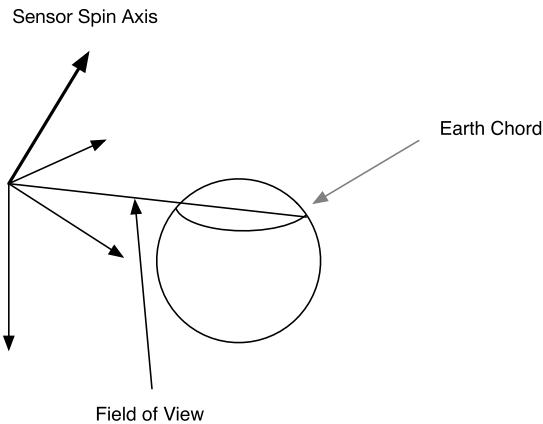


Figure 22.5 Earth-sensor geometry diagram.

22.5.2 Actuators

This design uses four reaction wheels mounted in a pyramid about the yaw axis. All four wheels run at the same time but only three are needed for three-axis control. Reaction wheels make sense in this application because the spacecraft must maneuver about both the pitch and yaw axes, and since the maneuvers are cyclic, the momentum can be transferred back and forth between the core and the reaction wheels without the use of any consumables.

Thrusters will be required to perform orbit-adjust maneuvers since misalignments and thrust mismatches in the orbit-adjust thrusters may be significant. Thrusters will also be needed for attitude control during maneuvers.

Momentum unloading can be performed by thrusters or by magnetic torquers. The torque produced by a torquer of dipole M is

$$T = M^\times B \quad (22.16)$$

where

$$M^\times = \begin{bmatrix} 0 & -M_z & M_y \\ M_z & 0 & -M_x \\ -M_y & M_x & 0 \end{bmatrix} \quad (22.17)$$

The above matrix cannot be inverted; consequently, it is not possible to realize a three-axis torque demand with magnetic torquers. However, if B changes it will be possible on average to satisfy an average three-axis demand. A simple algorithm to match the desired torque to the dipole is

$$M = -\frac{B^\times T}{|B|^2} \quad (22.18)$$

This is the least-squares solution to the desired torque. This requires three torquers.

An alternative is to use a dot-product algorithm. Assume we have two torquers. The torque axis of torquer i is

$$u_i = \frac{m_i \times B}{|m_i \times B|} \quad (22.19)$$

where small letters imply unit vectors. The dot product is taken between the torque demand with this vector. The torquer i is then energized to produce this torque. The dot product of the remainder of the torque is taken with the second torquer.

22.6. Attitude determination

22.6.1 Roll

A conical scanning Earth sensor returns the Earth chordwidth as the measurement. The chordwidth is related to the nadir angle, the angle between the spin axis of the sensor and the nadir vector, by the relationship

$$\cos \rho = \sin \eta \sin \gamma \cos \frac{\Omega}{2} + \cos \gamma \cos \eta \quad (22.20)$$

where r is the Earth angular radius, η is the nadir angle, Ω is the chordwidth, and γ is the cant angle between the spin-axis of the sensor and the boresight. Using the sum-of-angles trigonometric identity this becomes

$$\cos \left(\eta + \tan^{-1} \left(\tan \gamma \cos \frac{\Omega}{2} \right) \right) = \frac{\cos \rho}{\sqrt{(\sin \gamma \cos \frac{\Omega}{2})^2 + \cos^2 \gamma}} \quad (22.21)$$

The nadir angle is related to roll through the spherical triangle identity

$$\cos\left(\theta_z + \delta - \frac{\pi}{2}\right) = \cos\theta_y \cos\eta \quad (22.22)$$

where δ is the offset between the spin-axis and the spacecraft pitch-axis and θ_y is the spacecraft pitch angle.

22.6.2 Pitch

The pitch measurement is just the average of the Earth's rise and fall times with respect to a fixed reference in the sensor.

22.6.3 Sun-sensor eye preprocessing

The output from each Sun sensor eye is an analog voltage. We need to recover the value of the dot product between the Sun unit vector and the eye boresight. The response of the sensor can be written as

$$v = \sum_{k=0}^N a_k x^k \quad (22.23)$$

where $x = \cos\theta$. If

$$a_k \ll a_1 \quad k > 1 \quad (22.24)$$

then x can be extracted from v using a Newton–Raphson algorithm.

22.6.4 Solar-array pitch

The solar-array pitch measurement is computed from the Sun sensor eyes with components in the $\pm x$ directions. The algorithm is

$$\theta = \sin^{-1}\left(\frac{\Delta m}{\sqrt{2}}\right) \quad (22.25)$$

where Δm is the difference between the $+x$ and $-x$ eye measurements.

This is

$$u_y = \begin{bmatrix} 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & -\theta_y \\ 0 & 1 & \theta_x \\ \theta_y & -\theta_x & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_x \\ s_y \\ s_z \end{bmatrix} \quad (22.26)$$

where s is the Sun vector.

22.6.5 Yaw

The measurement from the $\pm\gamma$ eyes is of the γ component of the Sun vector in the array frame. Solving for yaw in terms of u_γ and s gives

$$u_\gamma = s_\gamma \cos \theta - s_x \sin \theta + s_z \theta_x \quad (22.27)$$

Using the sum of angles trigonometric identity

$$\theta = \sin^{-1} \left(\frac{u_\gamma - s_z \theta_x}{\sqrt{s_x^2 + s_\gamma^2}} \right) - \tan^{-1} \left(\frac{s_\gamma}{-s_x} \right) \quad (22.28)$$

The u_γ component is just

$$u_\gamma = \frac{\Delta m}{\sqrt{2}} \quad (22.29)$$

where Δm is the difference between the $+\gamma$ and $-\gamma$ eyes.

Eq. (22.28) shows the sensitivity of the yaw measurement to geometry. When s_x and $s_\gamma = 0$ the Sun lies along the z -axis and the Sun-sensor measurement is completely insensitive to yaw. When the x and γ components are small, the measurement will be sensitive to yaw but the noise will be amplified due to the geometry. Consequently, there will be some regions around the spacecraft at noon and midnight where it will be difficult to get good yaw measurements from the array-mounted Sun sensors. In these regions, gyros may be used or yaw estimated from roll and spacecraft momentum.

22.7. Control

22.7.1 Reaction-wheel loop

Reaction wheels are a motor with a flywheel attached. The motor base is attached to the spacecraft. A torque applied to the spacecraft by the motor causes the wheel to rotate in the other direction. The reaction wheel will experience friction due to bearings, windage, and other sources. The friction can be modeled as

$$\begin{aligned} T_F &= -\text{sgn}(\Omega) T_C - c\Omega & |\Omega| \geq \Omega_L \\ T_F &= \left(-\left(\frac{T_C + c\Omega_L}{\Omega_L} \right) \right) \Omega & |\Omega| < \Omega_L \end{aligned} \quad (22.30)$$

where sgn gives the sign of the argument, Ω is the wheel speed, and ω_L is the wheel speed limit below which Coulomb friction, T_C , is ignored. A pure damping model is used at low speeds to ensure that the wheel speed damps to zero in a simulation.

Otherwise, an unrealistic limit cycle will be caused by finite-size numerical integration. The reaction torque on the spacecraft is

$$T_R = T_M - T_F \quad (22.31)$$

Consequently, without knowledge of the friction there will always be a wheel-speed-dependent error. The vector angular-acceleration demand on the wheels is

$$\dot{\Omega} = \frac{K_p \theta + K_D \dot{\theta} + K_I \int \theta}{J} \quad (22.32)$$

In this example, the wheels are being used as momentum actuators since the wheels do not have a current feedback loop. The acceleration demand must be distributed among the wheels. If three orthogonal wheels are used then each component is sent to one of the three wheels. However, if more than three wheels are used it will be necessary to distribute the three acceleration components among more than three wheels. An additional relationship must be introduced. It is required that

$$\dot{\Omega} = U \dot{\Omega}_W \quad (22.33)$$

where U is a $3 \times n$ matrix, where $n \geq 3$, of the unit vectors of the reaction-wheel spin axes.

We want $\dot{\Omega}_W$ chosen so that

$$J = \frac{1}{2} \dot{\Omega}_W^T \dot{\Omega}_W + \lambda^T (\dot{\Omega} - U \dot{\Omega}_W) \quad (22.34)$$

is minimized. λ is a 3-vector of Lagrange multipliers that adjoin the constraint equations to the scalar cost equation. Taking derivatives of this equation with respect to $\dot{\Omega}_W$ and λ gives the two vector equations

$$\dot{\Omega} = U \dot{\Omega}_W \quad (22.35)$$

$$\dot{\Omega}_W = U^T \lambda \quad (22.36)$$

An alternative approach is to command wheel-speed changes instead of torques. This can be done because the equations of motion (neglecting Euler coupling) are

$$I \ddot{\theta} + J(\dot{\Omega} + \dot{\omega}) = T \quad (22.37)$$

If we make

$$J \dot{\Omega} = K_p \theta + K_D \dot{\theta} + K_I \int \theta \quad (22.38)$$

then we can use tachometer loops for each wheel to control the spacecraft. The tachometer drives the wheel speed to a commanded wheel speed. The commanded wheel speed is a function of the desired attitude correction.

22.7.2 Attitude loop

Since there will be steady external disturbances on the spacecraft, the attitude loops must contain integral action. A general form for a single-axis proportional-integral-differential (PID) controller is

$$T = K \left(1 + \tau_r \left(\frac{s}{\tau_f s + 1} \right) + \frac{1}{\tau_i s} \right) \quad (22.39)$$

where K is the forward gain, τ_r is the rate time constant, τ_f is the rate filter time constant, and τ_i is the integral time constant. This is generally implemented in a digital control system. If multirate sampling is available, it may be advantageous to run the rate filter at a higher rate than the rest of the controller.

22.7.3 Solar-array control

The solar-array pitch measurements are translated into solar-array steps, and the step demand is fed to the solar-array stepping motor. The stepping motor acts like a moderately damped second-order system.

Example 22.1 shows an attitude control simulation. The momentum-management system is not included. The simulation first calls the setup script. It designs PID controllers for each axis and a PI controller for the tachometer loops. The wheels do not have current feedback so a torque demand becomes a wheel-speed demand for the wheels. The tachometer loop controls the wheel speed. Changes in wheel speed translate into changes in body rates, thus controlling the spacecraft. This can be shown in a one-dimensional example.

Assume you have a dynamical system composed of two equations for the pitch loop of a momentum-bias spacecraft.

$$T = I\ddot{\theta} + J\dot{\Omega} \quad (22.40)$$

$$T_w = J\dot{\Omega} \quad (22.41)$$

where I is the pitch inertia, J is the momentum-wheel inertia, T is an external disturbance torque, θ is the angle that you want to control, Ω is your momentum-wheel speed, and T_w is the torque on the wheel produced by an electric motor connecting the wheel to the spacecraft. You have a measurement of Ω and θ but not T_w . How do we connect the two? Write

$$J\dot{\Omega} = I(2\zeta\sigma\dot{\theta} + \sigma^2\theta) \quad (22.42)$$

where ζ is the damping ratio and σ is the undamped natural frequency. Our first dynamical equation is now

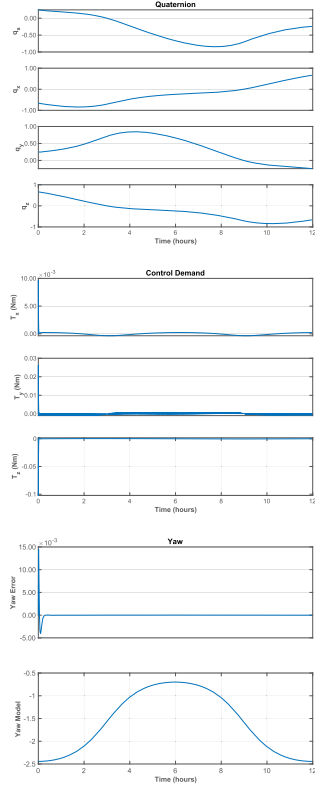
$$T = I\ddot{\theta} + I(2\zeta\sigma\dot{\theta} + \sigma^2\theta) \quad (22.43)$$

which is a damped second-order system. Divide by I to clarify the equation

```

1 %% Run the setup script
2 SunNadirSetup;
3
4 %% Initial conditions
5 [q,angleSA,w] = ICSN( xNoSun, wo, beta, alpha );
6 angleSAC = [angleSA;angleSA];
7 x = [q;angleSAC;w+wDelta:[0;0];[0;0;0;0]];
8
9 rW = zeros(6,1);
10 roll = 0;
11 pitch = 0;
12 yaw = 0;
13 uSunI = [cBeta; 0; sBeta];
14 mSS = SSOutput( angleSAC, nSA, uSSInSAF,
15     12B(q,uSunI), eyeSSCoeff, quantSSA );
16 xP = zeros(length(x)+5,nSim);
17
18 for k = 1:nSim
19     %% Attitude quaternion
20     qECIToBody = x(1:4);
21
22     %% Ephemeris
23     rNadir = rOrbit(-sin(alpha):0:cos(alpha));
24     uSunB = 12B( qECIToBody, uSunI );
25
26     %% RWA Tachometer
27     wTach = x(12:15);
28
29     %% Conical Scanning ESA
30     [chordwidth, scanAngle] = ESACS( qECIToBody,
31         rNadir, uESA, cantESAScan, pitchAxis );
32     rollIX = CW2Roll( scanAngle, cantESAScan, uESA,
33         uPitch, angEarth, chordwidth );
34     [~,j] = min(abs(rollIX));
35     roll = rollIX(j);
36     pitch = scanAngle;
37
38     %% Ephemeris Processing
39     [yawModel,sAngleModel,yawRateModel,saRateModel]
40     = SunNadir( xNoSun, wo, beta, alpha );
41
42     %% Solar array mounted sun sensors
43     mSS = SSOutput( x(5:6), nSA, uSSInSAF, uSunB,
44         eyeSSCoeff, quantSSA );
45     uSunL = [ -sin(alpha)*cBeta;-cos(alpha)*
46         cBeta ];
47     [yaw,sAPitch] = SSAD( cToDemSS, eyeSSCoeff,
48         uSunL, roll );
49
50     %% The attitude control loops
51     tC = -(cRoll*xRoll + dRoll*roll;...
52         cPitch*xPitch + dPitch*pitch;
53         cYaw*xYaw + dYaw*yaw);
54
55     xRoll = aRoll*xRoll + bRoll*roll;
56     xPitch = aPitch*xPitch + bPitch*pitch;
57     xYaw = aYaw*xYaw + bYaw*yaw;
58
59     %% Plotting
60     xP(:,k) = [x;tC;yaw;yawModel];
61
62     %% Solar Array Control
63     filteredSAPitch = kSAPitch*filteredSAPitch +
64         (1-kSAPitch)*sAPitch;
65     deltaAngle = sASize*fix(filteredSAPitch/
66         sASize);
67     angleSAC = angleSAC + [1;1]*deltaAngle;
68     dRHS.eSAC = angleSAC;
69
70     %% Torque demand to RWA accel demand
71     wDRWA = -aRWA*tC;
72
73     %% Integrate to get wheel speed demand
74     wRWAC = wRWAC + dT*wDRWA;
75
76     %% The RWA Tachometer Loops
77     wError = wTach - wRWAC;
78     dRWA = -dT*wError - cTL*xTL;
79     xTL = xTL + aTL*xTL + bTL*wError;
80
81     dRHS.rW(RWA) = dRWA;
82     x = RK4( @RHSSNP,x,dT,0,dRHS );
83
84     %% Update the orbit angle
85     alpha = alpha + woadT;
86 end
87
88 %% Plotting
89 t = (0:nSim-1)*dT;
90 TimeHistory(t,xP(1:4,:),{'q_s','q_x','q_y','q_z'},
91     'Quaternion');
92 TimeHistory(t,xP(7:9,:),{'\omega_x','\omega_y','\omega_z'},
93     'BodyRates');
94 TimeHistory(t,xP(12:15,:),{'\omega_1','\omega_2',
95     '\omega_3','\omega_4'}, 'Wheels');
96 TimeHistory(t,xP(16:18,:),{'T_x(Nm)','T_y(Nm)',
97     'T_z(Nm)' }, 'Control_Demand');
98 TimeHistory(t,xP(19:20,:),{'Err','Model'},'Yaw');

```



Example 22.1: Attitude control system simulation.

$$\frac{T}{I} = \ddot{\theta} + \zeta \sigma \dot{\theta} + \sigma^2 \theta \quad (22.44)$$

Our inner loop commands wheel speed.

$$T_w = Jk(\Omega_c - \Omega) \quad (22.45)$$

where k is the gain and Ω_c is the commanded wheel speed. The inner-loop dynamical equation is now

$$k\Omega_c = \dot{\Omega} + k\Omega \quad (22.46)$$

which is a damped first-order system. If k is high enough, that is the inner loop is very fast, the wheel speed will equal the commanded speed.

$$\Omega = \Omega_c \quad (22.47)$$

This is the “steady-state” response. How do we determine Ω ? Integrate Eq. (22.42)

$$\Omega = \left(\frac{I}{J}\right) \int (2\zeta \sigma \dot{\theta} + \sigma^2 \theta) \quad (22.48)$$

or

$$\Omega_c = \left(\frac{I}{J}\right) \sigma \left(2\zeta \dot{\theta} + \sigma \int \theta\right) \quad (22.49)$$

Our outer loop will control θ . Our inner loop will control Ω .

The function ICSN initializes the ECI to body quaternion and the solar-wing angles. β is the elevation of the Sun with respect to the orbit plane and α is the azimuth. The wing-mounted Sun sensors are modeled using SSOutput and SSAD. The conical scanning Earth sensor uses CW2Roll and ESACS.

Solar-array control uses a stepping motor. A first-order filter is applied to the pitch angle. The effect of the stepping motor can be seen in the γ -axis plot.

22.7.4 Momentum control

The spacecraft is nominally zero momentum in the absence of external disturbances. Of course, external disturbances will cause the momentum to grow and this momentum growth must be controlled. The reaction wheels can only exchange momentum with the body; hence some other type of actuator is required to remove momentum. The obvious choices are either thrusters or magnetic torquers. Magnetic torquers have the advantage that they do not produce a force on the spacecraft and therefore do not perturb the orbit, a major advantage if the satellite is used for navigation purposes. In addition, magnetic torquers can only remove momentum about two axes at one time although on average, if the magnetic-field direction in the body frame changes, they can remove momentum in all three axes.

The momentum in the spacecraft in the inertial frame is

$$H = A \left(I\omega + \sum_{i=1}^N u_i J(\Omega_i + u_i^T \omega) \right) \quad (22.50)$$

where ω is the body rate, Ω_i is the angular rate of the wheel with respect to the spacecraft, and u_i is the spin-axis of the i th wheel. The momentum control law is a proportional controller

$$T = -KH \quad (22.51)$$

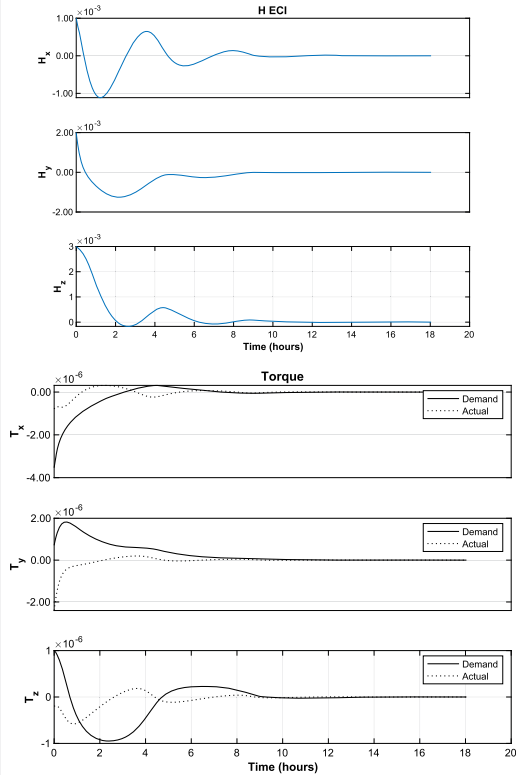
Momentum unloading is shown in Example 22.2. The simulation uses Euler integration where

$$\dot{x} = f(x, u) \quad (22.52)$$

```

1 %% Momentum Unloading using two torquers.
2 % The spacecraft is aligned with LVLH.
3 % The torquers are along roll and yaw.
4
5 e1 = [22000 pi/4 0 0 0 0];
6 p = Period(e1(1));
7 t = linspace(0,2*p,1000);
8 [r,v] = RVOrbGen(e1,t);
9 qLVLH = QVLH(r,v);
10 jD0 = Date2JD([2024 8 1]);
11 jD = jD0 + t/86400;
12 dT = t(2);
13 K = 0.001;
14
15 bECI = BDipole(r,jD);
16 bLVLH = QForm(qLVLH,bECI);
17 u1 = [1;0;0];
18 u2 = [0;0;1];
19 n = length(t);
20 xP = zeros(9,n);
21
22 hECI = [1;2;3]*1e-3;
23
24 % Simulate using Euler integration
25 for k = 1:n
26     torque = -K*hECI;
27     torque = QForm(qLVLH(:,k),torque);
28     um1 = Unit(Cross(u1,bLVLH(:,k)));
29     um2 = Unit(Cross(u2,bLVLH(:,k)));
30     torq1 = um1'*torque*um1;
31     torq2 = um2'*(torque-torq1)*um2;
32     torqECI = QForm(qLVLH(:,k),torq1+torq2);
33     xP(:,k) = [hECI;torque;torqECI];
34     hECI = hECI + dT*torqECI;
35 end
36
37 yL = {'H_x' 'H_y' 'H_z' 'T_x' 'T_y' 'T_z'};
38 iL = {'Demand','Actual'};
39     {'Demand','Actual'};
40     {'Demand','Actual'};
41 k = 1:3;
42 TimeHistory(t,xP(k,:),yL(k),'H_ECI');
43 k = 4:9;
44 TimeHistory(t,xP(k,:),yL(4:6),'Torque',...
45     {[1 4] [2 5] [3 6]},iL);

```



Example 22.2: Momentum unloading with two torquers in a 22 000-km orbit.

is replaced by

$$x_{k+1} = x_k + \Delta t f(x_k, u_k) \quad (22.53)$$

where δt is the time step.